



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Sistema de Visión Artificial
para el Mantenimiento de
Carreteras
Documentación Técnica**



Presentado por Marcos Gómez Vega
en Universidad de Burgos — 8 de julio de 2025
Tutor: Bruno Baruque Zano

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal y gestión del desarrollo	1
A.3. Estudio de viabilidad	5
Apéndice B Especificación de Requisitos	9
B.1. Introducción	9
B.2. Objetivos generales	9
B.3. Catálogo de requisitos	9
B.4. Especificación de requisitos	11
Apéndice C Especificación de diseño	23
C.1. Introducción	23
C.2. Diseño de datos	23
C.3. Diseño arquitectónico	28
C.4. Diseño procedimental	32
Apéndice D Documentación técnica de programación	43
D.1. Introducción	43
D.2. Estructura de directorios	43
D.3. Manual del programador	46

D.4. Compilación, instalación y ejecución del proyecto	47
D.5. Pruebas del sistema	47
Apéndice E Documentación de usuario	53
E.1. Introducción	53
E.2. Requisitos de usuarios	53
E.3. Instalación	54
E.4. Manual del usuario	55
E.5. Recomendaciones y soporte	63
Apéndice F Anexo de sostenibilización curricular	65
F.1. Introducción	65
F.2. Conciencia del impacto social y ambiental	65
F.3. Enfoque sistémico y diseño inclusivo	66
F.4. Participación y aprendizaje transversal	66
F.5. Ética profesional y compromiso personal	66
F.6. Conclusión	67
Bibliografía	69

Índice de figuras

A.1. Vista en cuadrícula de Microsoft Planner con tareas agrupadas por Sprint.	4
A.2. Gráficos de progreso por Sprint y categoría de tarea.	5
C.1. Ejemplo nodo usuario base de datos	25
C.2. Ejemplo nodo incidencia base de datos	26
C.3. Ejemplo nodo DEvice_tokens base de datos	27
C.4. Ejemplo firebase storage	27
C.5. Diagrama de componentes	30
C.6. Diagrama de despliegue	31
C.7. Diagrama de flujo Login	33
C.8. Diagrama de secuencia UML Login	34
C.9. Diagrama de flujo Registro	36
C.10. Diagrama de secuencia UML Registro	37
C.11. Diagrama de flujo crear y enviar incidencia	38
C.12. Diagrama de secuencia UML crear y enviar incidencia	39
C.13. Diagrama de flujo ver lista de incidencias	40
C.14. Diagrama de secuencia UML ver lista de incidencia	41
D.1. Matriz confusión MobileNetV2	49
D.2. Matriz confusión EfficientNetB2	50
E.1. Primer paso: descargar la app desde el repositorio	54
E.2. Paso para instalar la aplicación en Android	54

Índice de tablas

A.1. Comparación de precios de servicios Firebase.	6
B.1. CU-01 Registro de usuario.	11
B.2. CU-02 Inicio de sesión del usuario.	12
B.3. CU-03 Cambio de la contraseña del usuario.	13
B.4. CU-04 Cambio del correo del usuario.	14
B.5. CU-05 Cambio del avatar del usuario.	15
B.6. CU-06 Cierre de sesión del usuario.	16
B.7. CU-07 Eliminar la cuenta del usuario.	17
B.8. CU-08 Registro de una incidencia.	18
B.9. CU-09 Envío de una incidencia.	19
B.10.CU-10 Visualizar incidencias del usuario.	20
B.11.CU-11 Ver detalles de una incidencia.	21
B.12.CU-12 Cambio de tipo de incidencia.	22
D.1. Comparativa de métricas para MobileNetV2 y EfficientNetB2	52

Apéndice A

Plan de Proyecto Software

A.1. Introducción

En este apéndice se detalla el plan de ejecución del proyecto software, abordando la planificación temporal estructurada en iteraciones (Sprints), las herramientas utilizadas para la gestión de tareas, y un estudio de viabilidad previo, tanto económico como legal, que garantiza la viabilidad del desarrollo de la aplicación propuesta.

A.2. Planificación temporal y gestión del desarrollo

El desarrollo del proyecto ha seguido un enfoque ágil, utilizando una planificación iterativa basada en **Sprints de una semana**. Esta metodología ha permitido adaptarse a los cambios de forma flexible y progresiva, favoreciendo una mejora continua y una entrega incremental de funcionalidades.

La planificación y el seguimiento se han realizado principalmente con **Microsoft Planner** para la organización de tareas y **GitHub Issues** para registrar problemas técnicos, vincular commits y hacer seguimiento del progreso real. Cada tarea está asociada a un Sprint, marcada con su prioridad (Alta, Media o Baja) y asignada con fechas estimadas de inicio y fin.

Estructura del desarrollo

El proyecto se ha dividido en **8 Sprints principales**, distribuidos de la siguiente manera:

- **Sprint 1 - Preparación y análisis (22/04/2025 - 25/04/2025):**
 - Investigación tecnológica (frameworks, librerías, IA).
 - Estudio de requisitos funcionales y no funcionales.
 - Configuración de entorno, repositorio y base del proyecto.
- **Sprint 2 - Estructura inicial de la App y backend de las primeras pantallas (26/04/2025 - 01/05/2025):**
 - Crear la base de datos SQLite.
 - Crear el frontend y el diseño de la pantalla de login y crear cuenta.
 - Crear el backend y la estructura inicial de la pantalla login y crear cuenta.
- **Sprint 3 - Desarrollo inicial de las pantallas principales de la app (02/05/2025 - 05/05/2025):**
 - Diseño de pantallas principales y navegación entre ellas.
 - Integración con la base de datos.
- **Sprint 4 - Reestructuración de la App (05/05/2025 - 12/05/2025):**
 - Cambiar la estructura de la App a navigation drawer.
 - Modificar todo el backend de las pantallas para adaptarlas a esta estructura nueva.
 - Modificar el diseño para esta estructura nueva.
- **Sprint 5 - Implementación de Firebase (13/05/2025 - 16/05/2025):**
 - Eliminar la base de datos SQLite y usar Firebase.
 - Usar Firebase para la autenticación de los usuarios.
 - Modificar el backend de las pantallas para usar Firebase.
- **Sprint 6 - Añadir nuevas pantallas y funcionalidades a la App (17/05/2025 - 26/05/2025):**

- Añadir el diseño y la implementación de la pantalla de configuración.
 - Crear la implementación para ordenar la lista de incidencias.
 - Configurar cambio a modo oscuro de la App.
 - Configurar cambio de correo del usuario.
 - Configurar cambio de contraseña del usuario.
 - Configurar cambio de avatar del usuario.
 - Configurar logout.
- **Sprint 7 - Añadir notificaciones y página web para usuario administrador (27/05/2025 - 24/06/2025):**
- Usar Firebase para notificaciones del usuario.
 - Crear una página web para que el usuario admin pueda ver todas las incidencias.
 - Configurar el cambio de idioma de la App.
 - Usar calle, ciudad y país en vez de coordenadas.
- **Sprint 8 - Reestructurar el repositorio e incluir el modelo final de IA (25/06/2025 - 06/07/2025):**
- Crear o cargar en la app el modelo definitivo de IA.
 - Reorganizar el directorio de GitHub.

Es importante destacar que las fechas de entrega y finalización de cada Sprint pueden verificarse de forma transparente mediante el historial de commits en GitHub y el seguimiento de tareas en Microsoft Planner. Esto permite mantener una trazabilidad clara y evidencia documental del progreso y cumplimiento de los plazos establecidos.

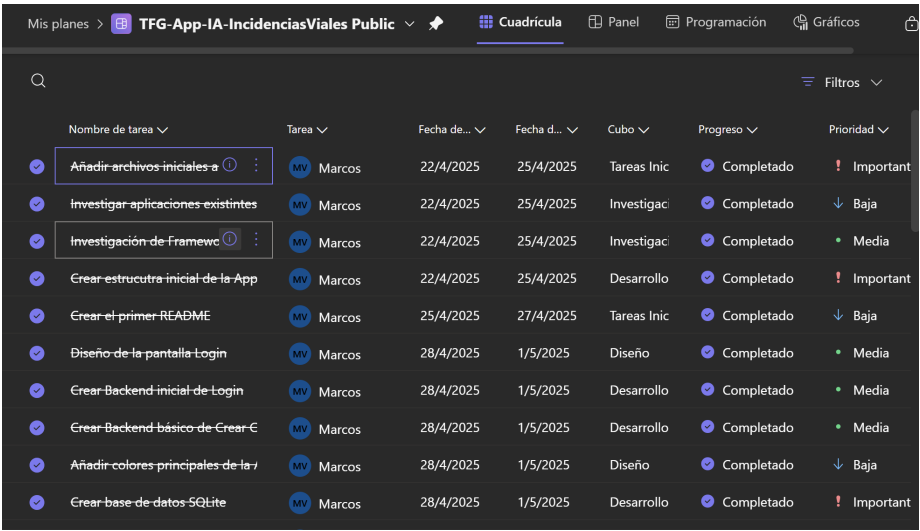
Gestión y trazabilidad

Cada tarea creada en Planner está vinculada a un **Issue en GitHub**, y los commits relevantes están referenciados en dicha tarea, permitiendo mantener una trazabilidad precisa entre la planificación y el desarrollo real.

Además, se utilizó la vista de cuadrícula de Planner para una visualización clara del estado de cada tarea, así como el gráfico de progreso general por Sprint.

Puedes consultar el planificador en el siguiente enlace (requiere acceso autorizado): **Planner**^[2]

A continuación se muestran capturas del panel de tareas y del resumen gráfico de evolución:



The screenshot shows the Microsoft Planner interface in grid view. The top navigation bar includes 'Mis planes > TFG-App-IA-IncidenciasViales Public', a star icon, and tabs for 'Cuadrícula', 'Panel', 'Programación', and 'Gráficos'. A search bar and a 'Filtros' dropdown are on the right. The main area displays a table of tasks with columns for 'Nombre de tarea', 'Tarea', 'Fecha de...', 'Fecha d...', 'Cubo', 'Progreso', and 'Prioridad'. Each task row includes a checkmark, a task name, a person icon (MV Marcos), start and end dates, a cube icon, a progress indicator, and a priority level.

	Nombre de tarea	Tarea	Fecha de...	Fecha d...	Cubo	Progreso	Prioridad
✓	Añadir archivos iniciales a	MV Marcos	22/4/2025	25/4/2025	Tareas Inic	Completado	Important
✓	Investigar aplicaciones existentes	MV Marcos	22/4/2025	25/4/2025	Investigaci	Completado	Baja
✓	Investigación de Framework	MV Marcos	22/4/2025	25/4/2025	Investigaci	Completado	Media
✓	Crear estructura inicial de la App	MV Marcos	22/4/2025	25/4/2025	Desarrollo	Completado	Important
✓	Crear el primer README	MV Marcos	25/4/2025	27/4/2025	Tareas Inic	Completado	Baja
✓	Diseño de la pantalla Login	MV Marcos	28/4/2025	1/5/2025	Diseño	Completado	Media
✓	Crear Backend inicial de Login	MV Marcos	28/4/2025	1/5/2025	Desarrollo	Completado	Media
✓	Crear Backend básico de Crear E	MV Marcos	28/4/2025	1/5/2025	Desarrollo	Completado	Media
✓	Añadir colores principales de la i	MV Marcos	28/4/2025	1/5/2025	Diseño	Completado	Baja
✓	Crear base de datos SQLite	MV Marcos	28/4/2025	1/5/2025	Desarrollo	Completado	Important

Figura A.1: Vista en cuadrícula de Microsoft Planner con tareas agrupadas por Sprint.



Figura A.2: Gráficos de progreso por Sprint y categoría de tarea.

A.3. Estudio de viabilidad

El estudio de viabilidad analiza si el proyecto puede llevarse a cabo de forma realista con los recursos disponibles. Para ello, se aborda tanto la viabilidad económica como la legal, fundamentales para un desarrollo seguro, sostenible y conforme a la legislación vigente.

Viabilidad económica

El proyecto ha sido desarrollado de forma individual, sin financiación externa, por lo que no ha requerido costes directos. Para el entorno de desarrollo y despliegue en fase beta, se han utilizado servicios gratuitos.

- **Firebase** (plan gratuito): hosting, base de datos, autenticación.

- **GitHub:** control de versiones y gestión de Issues.
- **Herramientas de desarrollo open source:** frameworks, librerías.

Firebase proporciona un conjunto de servicios gratuitos suficientes para una primera versión funcional. Si el proyecto escala y requiere más recursos, será necesario adoptar uno de los planes de pago.

Característica	Plan Gratuito	Plan de Pago
Almacenamiento	1 GB gratis	0.12 €/GB adicional
Base de datos	100 conexiones simultáneas	Hasta 200,000 conexiones
Autenticación	Hasta 50k usuarios/mes	Escalable con MAU

Tabla A.1: Comparación de precios de servicios Firebase.

También se contemplan alternativas como **AWS Free Tier**, que ofrece servicios gratuitos durante 12 meses con ciertos límites, siendo una opción viable en caso de migración.

Viabilidad legal

Desde el inicio se han respetado las normativas sobre desarrollo de software, uso de datos y propiedad intelectual. El proyecto no maneja datos personales sensibles, aunque se ha previsto que en caso de ampliación futura se cumpla estrictamente con:

- **RGPD** (UE 2016/679): Reglamento General de Protección de Datos.
- **LOPDGDD** (Ley Orgánica 3/2018): Protección de datos personales en España.

Se ha implementado una restricción desde el diseño: **no se permite subir imágenes de personas**, solo de incidencias reales (baches, señales, desperfectos). Esto evita conflictos legales relacionados con la privacidad.

Todas las herramientas utilizadas tienen licencias compatibles con proyectos educativos o personales, incluyendo Firebase, librerías de frontend/backend e inteligencia artificial.

En resumen, el proyecto es viable técnicamente, económicamente sostenible en su fase actual y legalmente conforme con la normativa vigente.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este apéndice recoge todos los requisitos de la aplicación, desde los funcionales hasta los no funcionales, los cuales estarán más detallados junto con sus respectivos casos de uso. Esta información sirve como principal base de diseño de la aplicación, así como de validación del correcto funcionamiento de esta.

B.2. Objetivos generales

Los objetivos generales del proyecto están en el Capítulo 2, donde se explican los objetivos principales de la aplicación, así como los objetivos técnicos de esta. En los siguientes apartados se explican con más detalle los requisitos funcionales y no funcionales de la aplicación, especialmente los funcionales, a través de los casos de uso.

B.3. Catálogo de requisitos

Gestión de usuarios:

- **RF-01:** Registro con correo y contraseña.
- **RF-02:** Registro mediante cuenta de Google.
- **RF-03:** Inicio de sesión con credenciales.

- **RF-04:** Cambio de contraseña.
- **RF-05:** Cambio de correo electrónico.
- **RF-06:** Cambio de avatar.
- **RF-07:** Cierre de sesión.
- **RF-08:** Eliminación de cuenta.

Gestión de incidencias:

- **RF-09:** Crear una incidencia desde una foto.
- **RF-10:** Enviar incidencia a las autoridades.
- **RF-11:** Visualizar incidencias del usuario.
- **RF-12:** Ver detalles de una incidencia.
- **RF-13:** Seleccionar el tipo de incidencia si la IA no esta segura.
- **RF-12:** Visualizar un mapa con la ubicacion de la incidencias.

Interacción con la aplicación:

- **RF-13:** Acceder a configuración.
- **RF-14:** Acceder a la pantalla principal.
- **RF-15:** Acceder a lista de incidencias.
- **RF-16:** Acceder a guía o tutorial.
- **RF-17:** Compartir la aplicación.

Configuración:

- **RF-18:** Cambiar entre modo oscuro y claro.
- **RF-19:** Cambiar el idioma de la aplicación.
- **RF-20:** Activar o desactivar notificaciones.

B.4. Especificación de requisitos

A continuación, se describen algunos de los principales casos de uso que cubren los requisitos descritos anteriormente.

Gestion de Usuarios

CU-01	Registro de usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-01, RF-02
Descripción	El usuario puede crear una cuenta en la aplicación mediante correo electrónico y contraseña o mediante una cuenta de Google.
Precondición	El usuario no debe tener sesión iniciada.
Acciones	1. El usuario accede a la pantalla de Crear sesión. 2. Introduce los datos requeridos o crea una cuenta en la aplicación con una cuenta de Google. 3. Se valida la información introducida. 4. Se crea la cuenta y se accede a la pantalla principal.
Postcondición	El usuario queda registrado y autenticado.
Excepciones	Error en conexión, datos inválidos, correo ya registrado, Error al abrir la pantalla.
Importancia	Alta

Tabla B.1: CU-01 Registro de usuario.

CU-02	Inicio de sesión del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-03
Descripción	El usuario puede iniciar sesión en la aplicación mediante correo electrónico y contraseña o mediante una cuenta de Google o Facebook.
Precondición	El usuario no debe tener sesión iniciada.
Acciones	1. El usuario accede a la pantalla de Inicio de sesión. 2. Introduce los datos requeridos o selecciona el botón para iniciar sesión con una cuenta de Google/Facebook. 3. Se valida la información introducida. 4. Se accede a la pantalla principal.
Postcondición	El usuario es autenticado.
Excepciones	Error en conexión, datos inválidos, Error al abrir la pantalla.
Importancia	Alta

Tabla B.2: CU-02 Inicio de sesión del usuario.

CU-03	Cambio de la contraseña del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-04
Descripción	El usuario puede cambiar la contraseña asociada a una cuenta de la aplicación.
Precondición	El usuario debe tener sesión iniciada.
Acciones	1. El usuario accede a la aplicación. 2. El usuario accede al menú lateral pulsando el botón de “menú”. 3. El usuario accede a la pantalla de configuración pulsando el botón de “configuración”. 4. El usuario accede al diálogo de cambio de contraseña pulsando el botón de “cambiar contraseña”. 5. Introduce los datos requeridos para el cambio de contraseña. 6. Se valida la información introducida. 7. Se accede a la pantalla de configuración.
Postcondición	La contraseña del usuario se ha cambiado correctamente.
Excepciones	Error en conexión, datos inválidos, contraseña inválida, Error al abrir diálogo.
Importancia	Media

Tabla B.3: CU-03 Cambio de la contraseña del usuario.

CU-04	Cambio del correo del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-05
Descripción	El usuario puede cambiar el correo asociado a una cuenta de la aplicación.
Precondición	El usuario debe tener sesión iniciada.
Acciones	1. El usuario accede a la aplicación. 2. El usuario accede al menú lateral pulsando el botón de “menú”. 3. El usuario accede a la pantalla de configuración pulsando el botón de “configuración”. 4. El usuario accede al diálogo de cambio de correo pulsando el botón de “cambiar correo”. 5. Introduce los datos requeridos para el cambio de correo. 6. Se valida la información introducida. 7. El usuario verifica el nuevo correo accediendo al correo de verificación enviado desde la app. 8. Se accede a la pantalla de configuración.
Postcondición	El correo del usuario se ha cambiado correctamente, Error al abrir diálogo.
Excepciones	Error en conexión, datos inválidos, correo inexistente.
Importancia	Media

Tabla B.4: CU-04 Cambio del correo del usuario.

CU-05	Cambio del avatar del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-06
Descripción	El usuario puede cambiar el avatar asociado a una cuenta de la aplicación.
Precondición	El usuario debe tener sesión iniciada.
Acciones	1. El usuario accede a la aplicación. 2. El usuario accede al menú lateral pulsando el botón de “menú”. 3. El usuario accede a la pantalla de configuración pulsando el botón de “configuración”. 4. El usuario accede al diálogo de cambio de avatar pulsando el botón de “cambiar avatar”. 5. Introduce una nueva foto. 6. Se valida la información introducida. 7. Se accede a la pantalla de configuración.
Postcondición	El avatar del usuario se ha cambiado correctamente.
Excepciones	Error en conexión, datos inválidos, Error al abrir diálogo.
Importancia	Baja

Tabla B.5: CU-05 Cambio del avatar del usuario.

CU-06	Cierre de sesión del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-07
Descripción	El usuario puede cerrar sesión de la aplicación.
Precondición	El usuario debe tener sesión iniciada.
Acciones	1. El usuario accede a la aplicación. 2. El usuario accede al menú lateral pulsando el botón de “menú”. 3. El usuario puede cerrar sesión pulsando el botón de “cerrar sesión” o puede acceder a la pantalla de configuración pulsando el botón de “configuración”. 4. El usuario puede cerrar sesión pulsando el botón de “cerrar sesión”. 5. Se accede a la pantalla de Login.
Postcondición	Se habrá cerrado la sesión del usuario correctamente.
Excepciones	Error en conexión.
Importancia	Alta

Tabla B.6: CU-06 Cierre de sesión del usuario.

CU-07	Eliminar la cuenta del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-08
Descripción	El usuario puede eliminar la cuenta de la aplicación.
Precondición	El usuario debe tener sesión iniciada.
Acciones	1. El usuario accede a la aplicación. 2. El usuario accede al menú lateral pulsando el botón de “menú”. 3. El usuario accede a la pantalla de configuración pulsando el botón de “configuración”. 4. El usuario puede eliminar la cuenta pulsando el botón de “eliminar cuenta”. 5. El usuario deberá aceptar la alerta para eliminar la cuenta. 6. Se accede a la pantalla de Login.
Postcondición	Se habrá eliminado la cuenta del usuario con sus datos correctamente.
Excepciones	Error en conexión, Error al abrir diálogo.
Importancia	Media

Tabla B.7: CU-07 Eliminar la cuenta del usuario.

Gestión de Incidencias

CU-08	Registro de una incidencia
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-09
Descripción	El usuario puede crear una incidencia.
Precondición	El usuario debe tener sesión iniciada.
Acciones	1. El usuario accede a la cámara o a la galería pulsando al botón “tomar foto” para crear una incidencia. 2. El usuario acepta la foto. 3. Se valida la información introducida. 4. Se crea la incidencia. 5. Se accede a la pantalla principal.
Postcondición	La incidencia se crea.
Excepciones	Error en conexión, datos inválidos
Importancia	Alta

Tabla B.8: CU-08 Registro de una incidencia.

CU-09	Envío de una incidencia
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-10
Descripción	El usuario puede enviar una incidencia.
Precondición	El usuario debe tener sesión iniciada y una incidencia creada.
Acciones	1. El usuario envía la incidencia pulsando al botón “enviar incidencia”. 2. Se valida la información introducida. 3. Se envía la incidencia a las autoridades y se introduce en la base de datos. 4. Se accede a la pantalla principal.
Postcondición	La incidencia se envía y se guarda en la base de datos correctamente.
Excepciones	Error en conexión, datos inválidos
Importancia	Alta

Tabla B.9: CU-09 Envío de una incidencia.

CU-10	Visualizar incidencias del usuario
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-11
Descripción	El usuario puede visualizar todas sus incidencias.
Precondición	El usuario debe tener sesión iniciada y una incidencia enviada.
Acciones	1. El usuario accede a la pantalla con sus incidencias pulsando el botón “ver incidencias”. 2. Se muestra la pantalla con la lista de incidencias del usuario.
Postcondición	El usuario puede ver sus incidencias.
Excepciones	Error en conexión, Error al abrir la pantalla
Importancia	Media

Tabla B.10: CU-10 Visualizar incidencias del usuario.

CU-11	Ver detalles de una incidencia
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-12
Descripción	El usuario puede ver en detalle una incidencia.
Precondición	El usuario debe tener sesión iniciada y una incidencia enviada.
Acciones	1. El usuario accede a la pantalla con sus incidencias pulsando el botón “ver incidencias”. 2. El usuario selecciona la incidencia que desea consultar y pulsa el botón “Ver”. 3. Se muestra la pantalla con las características detalladas de la incidencia seleccionada.
Postcondición	El usuario puede ver las características de la incidencia seleccionada.
Excepciones	Error en conexión, Error al abrir la pantalla
Importancia	Media

Tabla B.11: CU-11 Ver detalles de una incidencia.

CU-12	Cambio de tipo de incidencia
Versión	1.0
Autor	Marcos Gómez Vega
Requisitos asociados	RF-13
Descripción	El usuario puede cambiar el tipo de incidencia.
Precondición	El usuario debe tener sesión iniciada y una incidencia creada.
Acciones	1. El usuario pulsa el botón donde este el tipo de incidencia. 2. El usuario elegirá que tipo de incidencia es 3. El usuario vera el nuevo tipo de incidencia
Postcondición	El usuario puede ver el nuevo tipo de incidencia.
Excepciones	Error en conexión, Error al abrir la pantalla
Importancia	Media

Tabla B.12: CU-12 Cambio de tipo de incidencia.

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se presenta la especificación del diseño de la aplicación, dividido en tres partes principales.

Primero, se detalla el diseño de datos, donde se explica la estructura de la base de datos basada en Firebase Realtime Database, cómo se organizan los nodos principales de usuarios e incidencias, y el almacenamiento de imágenes.

A continuación, se describe el diseño arquitectónico, incluyendo imágenes y diagramas que muestran cómo está estructurada la aplicación y sus componentes principales.

Finalmente, se expone el diseño procedimental, con diagramas que ilustran los procesos y cómo los usuarios realizan las diferentes acciones dentro de la aplicación.

El objetivo de este apéndice es ofrecer una visión clara y completa de la organización interna de la aplicación, facilitando la comprensión de su funcionamiento y diseño.

C.2. Diseño de datos

Tecnología y estructura general

La base de datos de mi aplicación usa Firebase Realtime Database. Esta estructura de base de datos, como su propio nombre indica, es una base

de datos en tiempo real que no es SQL. Esto me supuso un problema al principio, ya que tuve que acostumbrarme a usar este tipo de base, pero la ventaja de esta es que, al ser en tiempo real, cualquier persona puede ver los datos y las modificaciones al instante. Además, gracias a Firestore Database, las autoridades podrán obtener dichos datos en tiempo real; solo necesitarán una API para poder extraerlos.

Mi base de datos está estructurada en dos nodos principales: los usuarios, las incidencias y los tokens de los dispositivos, donde en cada nodo tendremos otros nuevos nodos con el UID de cada usuario o cada incidencia, y después sus respectivos datos que serán explicados más adelante.

Estructura del nodo Usuarios

En el nodo de usuarios tendremos los usuarios que han creado una cuenta. Aquí tendremos un nodo para cada usuario registrado con su UID único, que es la equivalencia a la clave primaria de SQL. Después, en cada nodo tendremos los siguientes datos:

- **Email:** Email del usuario con el que hacer el login a la aplicación.
- **Name:** Nombre de usuario que el usuario quiera para su aplicación.
- **Last login:** Día y la hora en la cual el usuario hizo su último login.
- **Avatar:** URL para extraer la foto del avatar del usuario, siempre que este quiera tener una foto de avatar; si no, estará el predeterminado.
- **Language:** Idioma en que el usuario quiere que este su aplicación, solo puede estar es que significa español y en que es inglés.

Un ejemplo de cómo queda este nodo desde la página web de Firebase es:

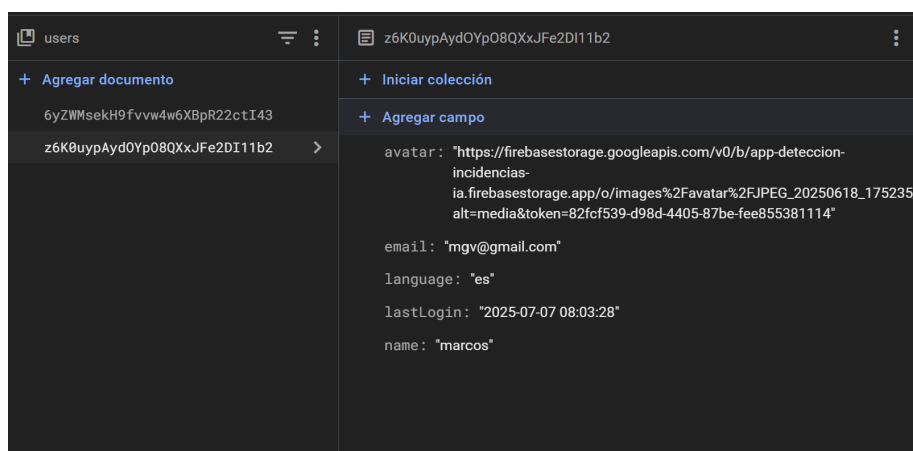


Figura C.1: Ejemplo nodo usuario base de datos

Estructura del nodo Incidencias

En el nodo de incidencias tendremos las incidencias que los usuarios han ido creando. Tendremos un nodo para cada incidencia con su respectivo UID único; después, en cada nodo tendremos los siguientes datos:

- **UID:** UID de la incidencia.
- **user__id:** UID del usuario que ha hecho esa incidencia.
- **incident__type:** Tipo de incidencia que ha detectado la IA.
- **status:** Estado de la incidencia.
- **coordinates:** Localización de la incidencia en coordenadas.
- **photo:** URL para extraer la foto de la incidencia.
- **date:** Día y hora en la cual la incidencia fue tomada.
- **error__percentage:** El porcentaje de lo segura que esta la IA a la hora de detectar el tipo de incidencia.
- **localitation** Calle, ciudad y país de la incidencia.

Un ejemplo de cómo queda este nodo desde la página web de Firebase es:

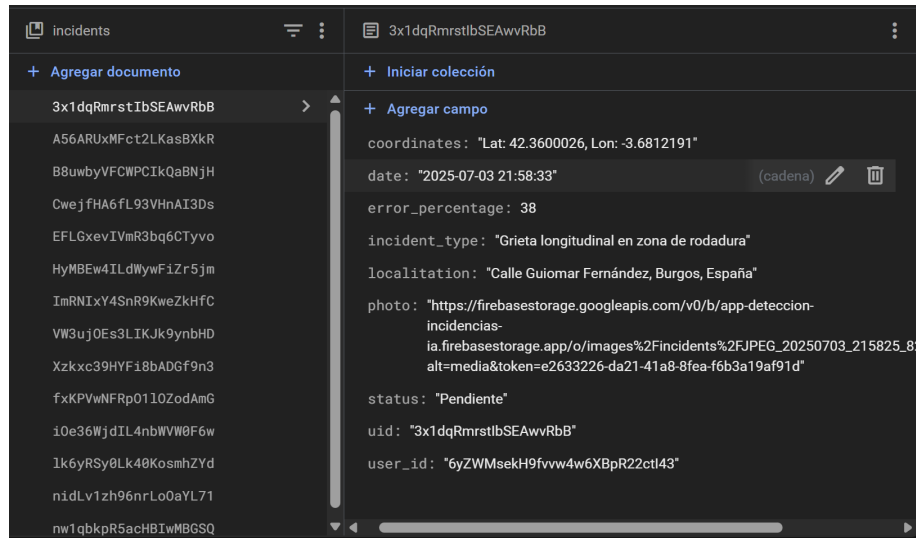


Figura C.2: Ejemplo nodo incidencia base de datos

Estructura del nodo Device_token

En el nodo token de dispositivos tendremos los distintos tokens de cada dispositivo que ha usado la app. Tendremos un nodo para cada dispositivo con su respectivo UID único; después, en cada nodo tendremos los siguientes datos:

- **notification_activated:** Booleano en el cual si es true el dispositivo admite notificaciones y si el false no.
- **timestamp:** Fecha en la que el dispositivo entre por primera vez en la app.
- **token:** Token único de cada dispositivo.
- **userId:** UID del usuario de la app, ya que un usuario puede tener varios dispositivos.

Un ejemplo de cómo queda este nodo desde la página web de Firebase es:

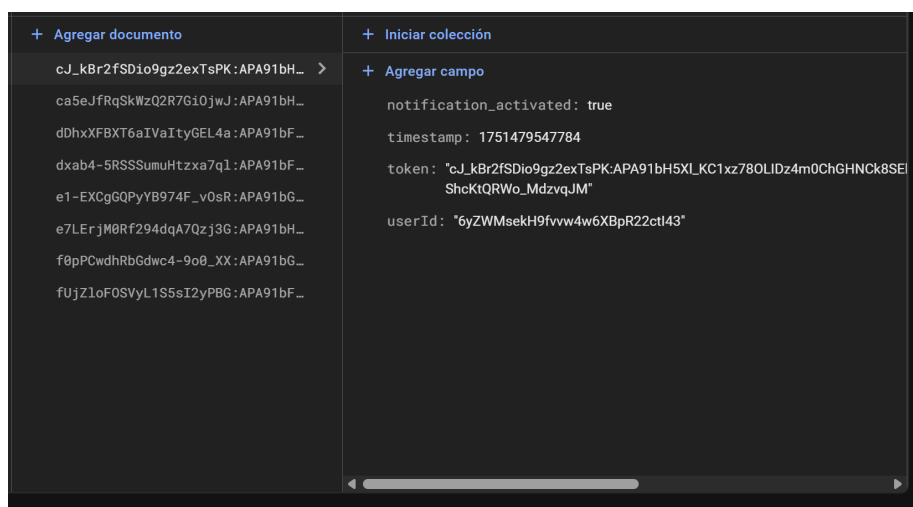


Figura C.3: Ejemplo nodo Device_tokens base de datos

Almacenamiento de imágenes

También he usado Firebase Storage para guardar todas las fotos de las incidencias, los recortes de la incidencia que detecte la IA y las fotos de los avatares, así estas fotos son disponibles para cualquier persona que tenga permiso.

Así, con estas imágenes y con la base de datos, las autoridades tendrán más claro cómo tratar las incidencias, y también estas imágenes podrán servir para entrenar a un modelo de IA que sea más exacto.

Un ejemplo de cómo queda el almacenamiento en Firebase Storage:

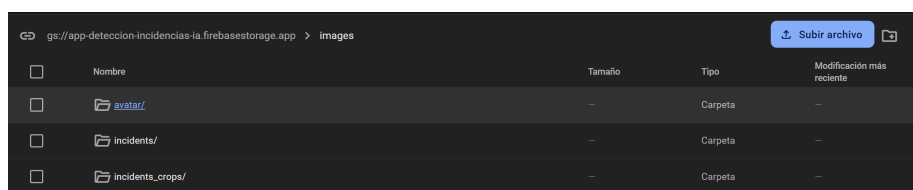


Figura C.4: Ejemplo firebase storage

C.3. Diseño arquitectónico

Visión general de la arquitectura

Para crear la aplicación he usado un modelo cliente-servidor, lo que significa que el usuario ejecuta la app en su móvil, que sería el cliente, y esta se conecta a un servidor, que en mi caso es Firebase por la simplicidad que te proporciona.

Para manejar todos los datos de tanto la autenticación, como la base de datos y el almacenamiento, usé Firebase, ya que es un servicio en la nube proporcionado por Google que hace de servidor. Esto me simplificaba mucho el backend de la aplicación, ya que Firebase tiene sus métodos implementados y solo hay que llamarlos y ejecutarlos. Así, cuando se cree un usuario o se cree una incidencia gracias al servidor, todo se mantendrá en la nube y no en el móvil, por lo que cualquier persona con acceso podrá extraer los datos.

De esta forma, la app es mucho más flexible y segura, porque toda la información importante está centralizada en la nube y no solo en el móvil. Además, permite que la app se mantenga sincronizada y actualizada sin que el usuario tenga que hacer nada extra.

Descripción de los layouts

La aplicación tiene varias pantallas principales que permiten al usuario interactuar con sus funcionalidades de forma sencilla e intuitiva. A continuación se describen los layouts más importantes:

Pantalla de inicio de sesión y registro:

Aquí el usuario puede crear una cuenta nueva o loguearse con sus credenciales. Esta pantalla está diseñada para ser clara y rápida, facilitando el acceso seguro a la aplicación.

Pantalla principal:

Es la primera que se muestra una vez que el usuario ha iniciado sesión. Desde este layout puedes crear una incidencia dando al botón de "Tomar foto" así como enviar dicha incidencia a las autoridades con "enviar Incidencias" y podrás ver tus incidencias dando al botón de "Ver incidencias" .

Pantalla de lista de incidencias:

Muestra todas las incidencias creadas por el usuario en forma de lista, permitiendo un acceso rápido y organizado.

Pantalla de detalle de incidencia:

Permite ver la información completa de una incidencia específica, incluyendo la foto tomada, tipo de incidencia, localización y estado.

Mapa:

Permite ver un mapa con todas tus incidencias con distintos colores para el estatus de cada incidencia. Si el zoom del mapa es cerca de 3km, entonces podrás ver las incidencias de otros usuarios.

Pantalla de configuración:

Donde el usuario puede ajustar opciones básicas de la aplicación, como notificaciones, idioma, cambio de correo ...

Diagramas de arquitectura

Para entender mejor cómo está organizada y funciona la aplicación, he creado dos tipos de diagramas: uno de componentes y otro de despliegue.

Diagrama de componentes

En el diagrama de componentes se puede ver cómo está dividida la app en diferentes paquetes y módulos, en este diagrama se ve cómo se relacionan entre ellos. Por ejemplo, hay un módulo para el inicio de sesión, otro para crear incidencias, otro para ver la lista, etc. Esto ayuda a tener una idea más clara de cómo está estructurada internamente la aplicación y como se conecta con otros paquetes como puede ser el firebase con sus módulos.

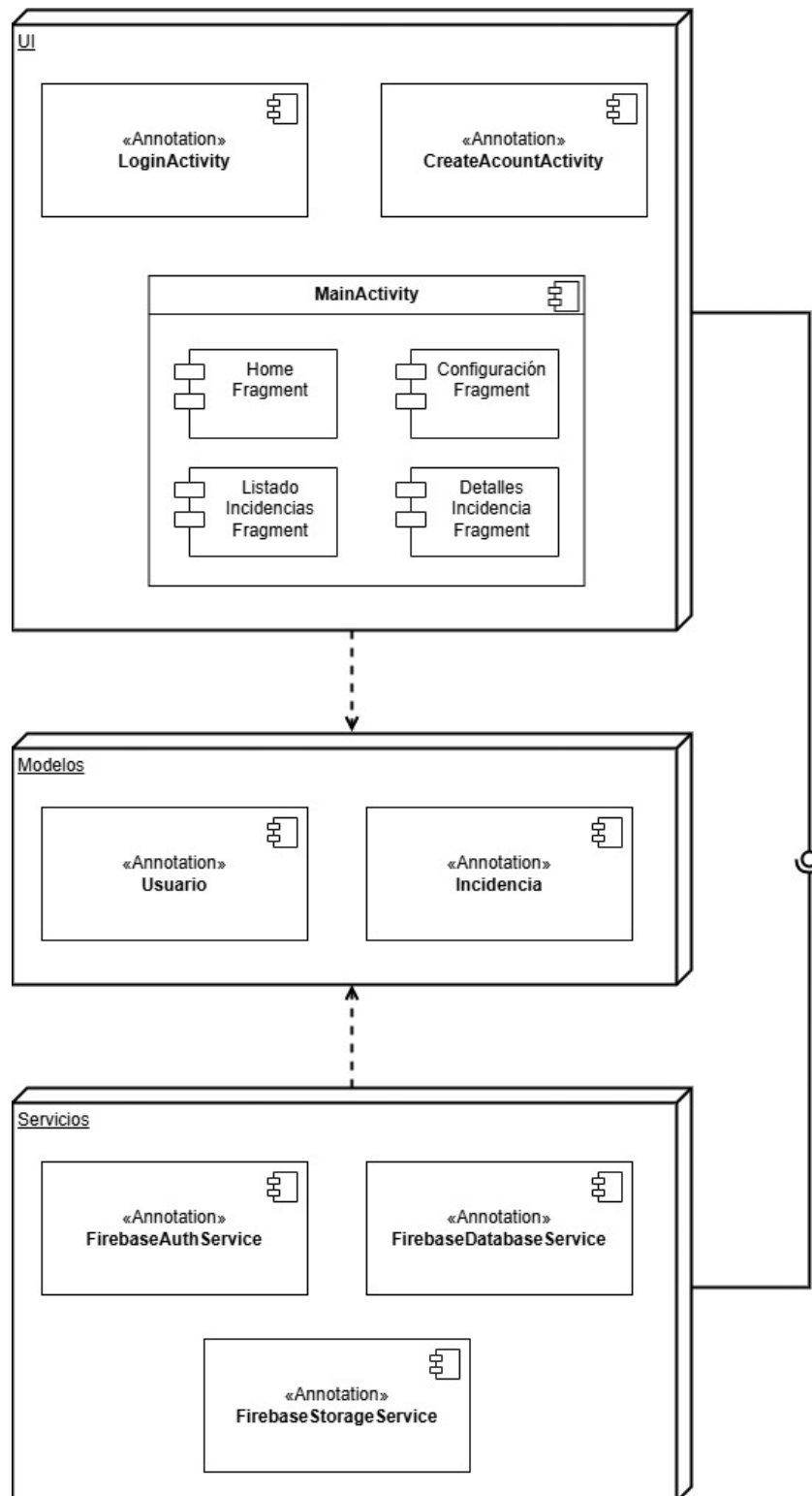


Figura C.5: Diagrama de componentes

Diagrama de despliegue

También he incluido un diagrama de despliegue, que muestra dónde se ejecuta cada cosa. Básicamente, la app corre en el móvil del usuario, pero todo lo importante (como los datos, el inicio de sesión y las fotos que se suben) se guarda y gestiona en Firebase, que está en la nube. Este diagrama ayuda a visualizar cómo el móvil se conecta a Internet para comunicarse con Firebase, y cómo se reparten las tareas entre el dispositivo del usuario y los servicios en línea.

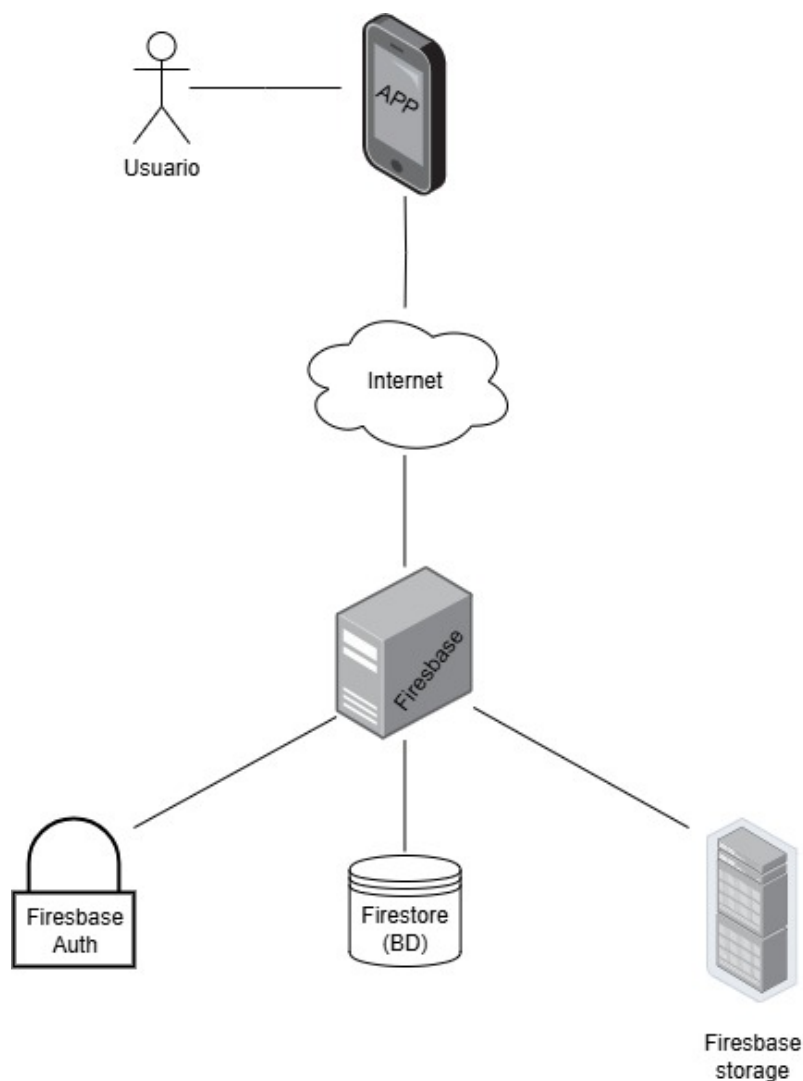


Figura C.6: Diagrama de despliegue

C.4. Diseño procedimental

En esta sección se explican los procesos clave que realiza el usuario dentro de la aplicación, entre ellos estarán los procesos de iniciar sesión, crear cuenta, crear una incidencia y consultar tus incidencias. Para cada uno de estas se mostrarán un diagrama de flujo que resume el proceso paso a paso y un diagrama de secuencia UML que muestra cómo interactúan los distintos componentes de la aplicación.

Login de usuario

Este apartado corresponde al requisito **RF-03**, el cual indica que un usuario registrado debe poder iniciar sesión en la aplicación introduciendo sus credenciales. En el caso de uso de dicho requisito se explican los pasos que tiene que seguir el usuario, así como los pre requisitos que necesita y el resultado de hacer bien dicho requisito.

El proceso de login empieza cuando el usuario introduce sus datos en el layout de login. Estos datos se envían al backend para ser validados mediante Firebase Authentication. Si las credenciales son correctas, el usuario accede correctamente a la aplicación y su UID se guarda para recuperar su información de la base de datos. Si los datos son incorrectos, se muestra un mensaje de error en pantalla.

Diagrama de flujo

A continuación se muestra el diagrama de flujo que representa los pasos principales que se siguen durante el proceso de login del usuario:

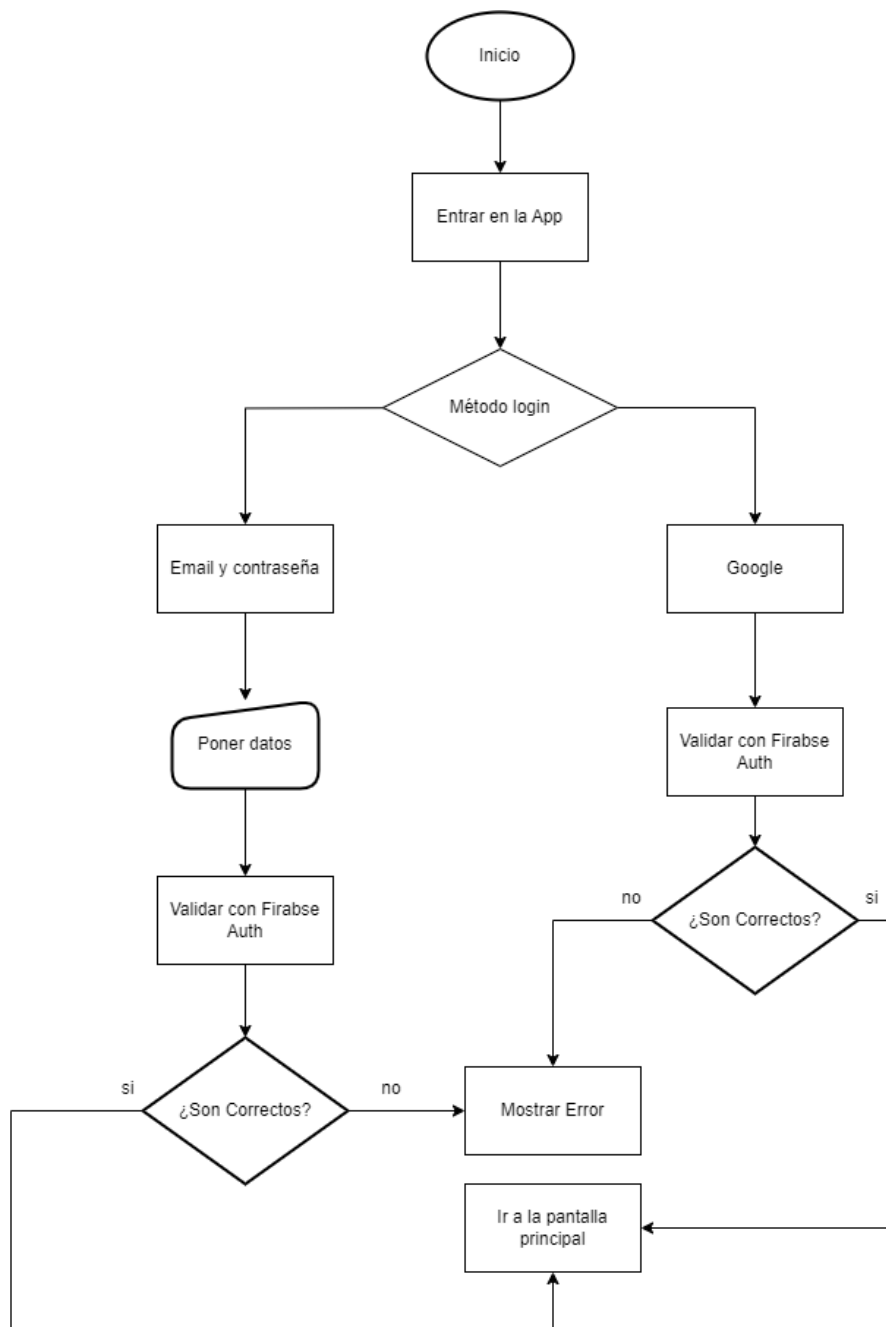


Figura C.7: Diagrama de flujo Login

Diagrama de secuencia UML

En el siguiente diagrama de secuencia se puede ver cómo interactúan los distintos componentes de la aplicación durante el proceso de login:

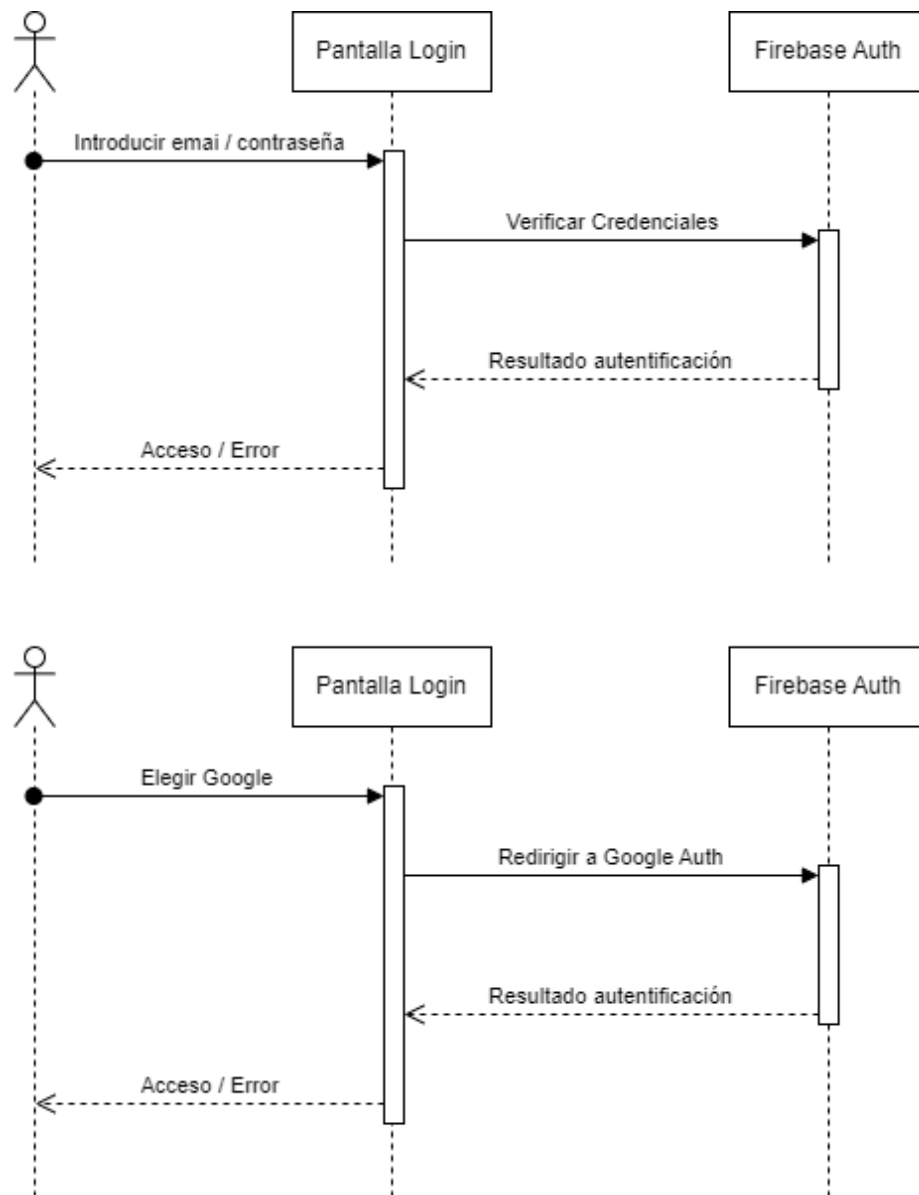


Figura C.8: Diagrama de secuencia UML Login

Creación de un usuario

Este apartado corresponde al requisito **RF-01**, que especifica que cualquier persona debe poder registrarse en la aplicación como nuevo usuario. En el caso de uso correspondiente explico los pasos que se siguen durante el proceso, así como las comprobaciones necesarias para que el registro se complete correctamente.

Para empezar, el usuario debe de entrar en la pantalla de creación de cuenta en la que debe introducir su nombre, correo electrónico, contraseña y la confirmación de esta. Además, es obligatorio aceptar las condiciones de uso antes de poder continuar. Una vez que el usuario introduce estos datos, desde la propia aplicación se realizan comprobaciones básicas: que los campos no estén vacíos, que las contraseñas coincidan y que se haya marcado la aceptación de condiciones. Si alguna de estas validaciones no se cumple, se muestra el mensaje de error correspondiente para que el usuario pueda corregirlo.

En caso de que los datos sean válidos, el siguiente paso consiste en registrar al nuevo usuario en la base de datos local. Si el registro se realiza con éxito, se notifica al usuario mediante un mensaje y se le redirige automáticamente a la pantalla de login. En esta pantalla, si el sistema detecta que ya existe una sesión activa, se accede directamente a la pantalla principal de la aplicación sin necesidad de volver a introducir los datos.

Diagrama de flujo

A continuación muestro el diagrama de flujo que resume de forma visual todos los pasos que se siguen durante el proceso de creación de una cuenta de usuario:

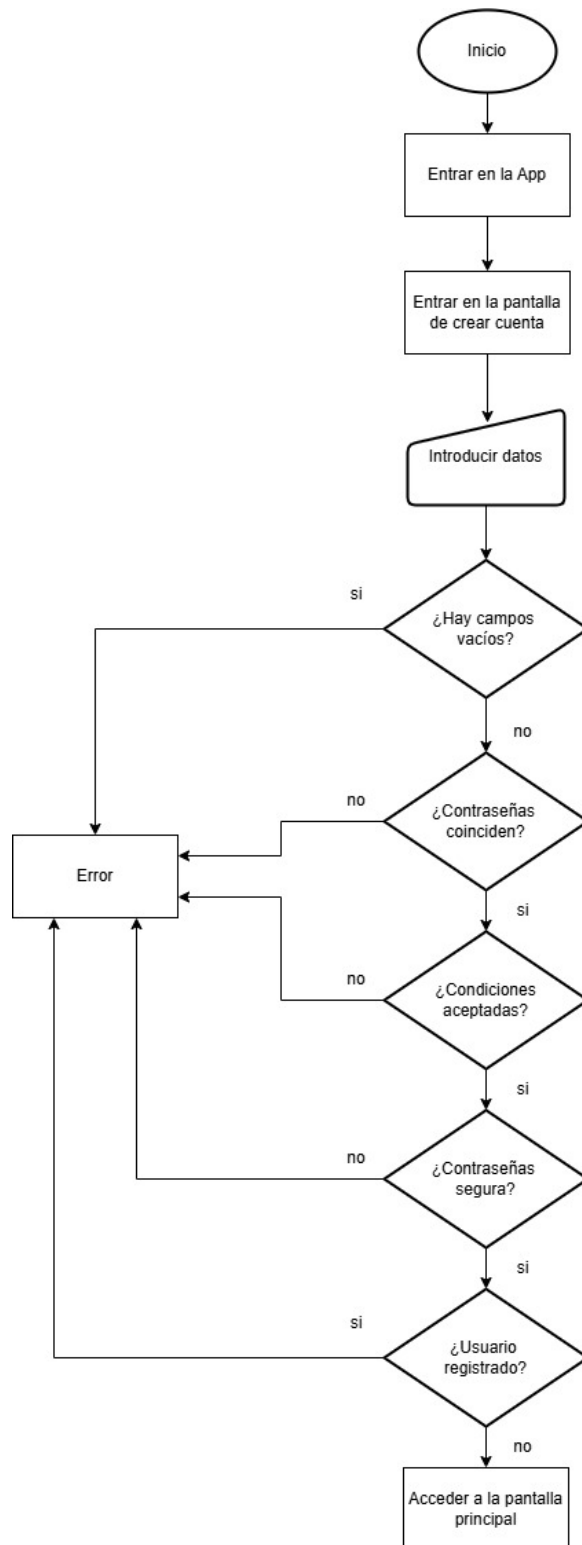


Figura C.9: Diagrama de flujo Registro

Diagrama de secuencia UML

En el siguiente diagrama de secuencia represento cómo interactúan entre sí el usuario, la interfaz gráfica, la base de datos y el sistema de autenticación de Firebase a lo largo del proceso de registro:

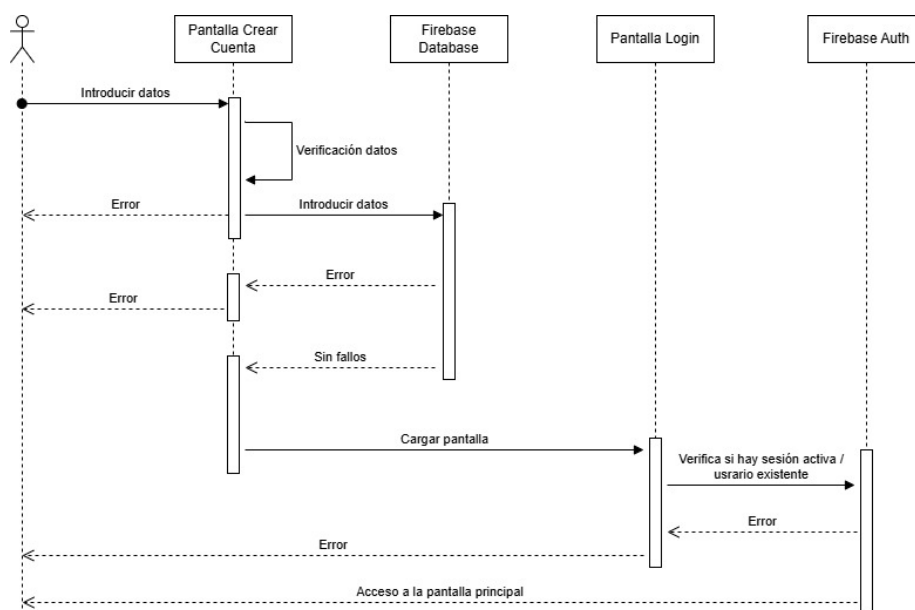


Figura C.10: Diagrama de secuencia UML Registro

Creación de una incidencia

Este apartado corresponde a los requisitos **RF-09** y **RF-10**, los cuales indican que un usuario registrado puede crear una incidencia y poder enviarlas a las autoridades, en los casos de uso de estos requisitos se explica los pasos que se debe de seguir para crear una incedncia y enviarlas a las autoridades, así como los pre requisitos que tiene que tener para poder crear y enviar dicha incidencia, y el resultado correcto de la creación y el envío de una incidencia.

Para crear una indecencia es indispensable tener una cuenta y estar registrado, después accederemos a la pantalla principal de la aplicación y ahí ya podremos crear una incidencia tomando una foto a dicha incidencia, esta sera clasificada por la IA y también sera geolocalizada, una vez tomada la foto se podrán ver los datos de esta incidencia, y si todo esta correcto se podrá enviar la iniciación a las autoridades y a la base de datos.

Diagrama de flujo

A continuación muestro el diagrama de flujo que resume de forma visual todos los pasos que se siguen durante el proceso de creación y el envío de una incidencia:

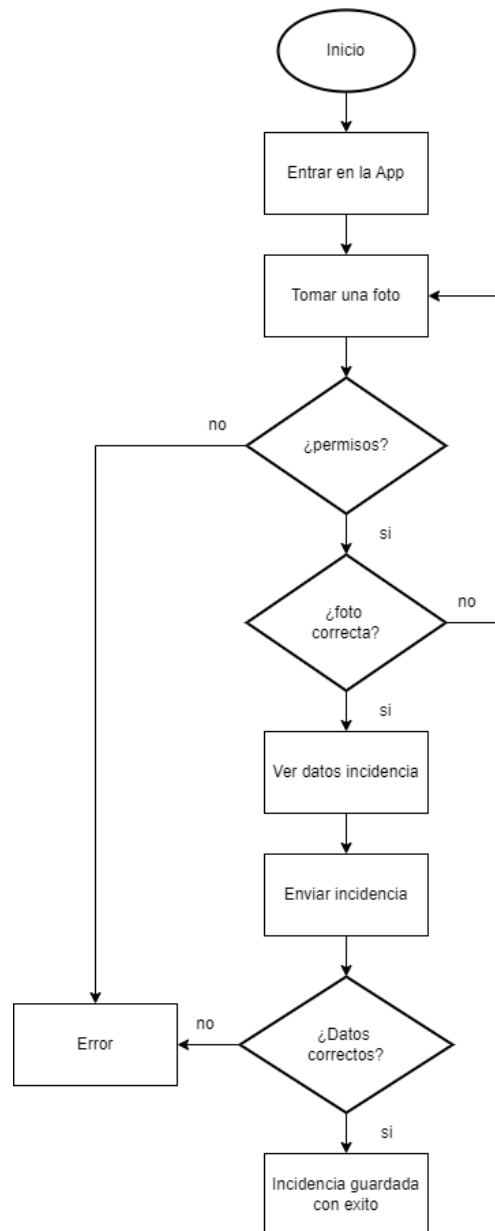


Figura C.11: Diagrama de flujo crear y enviar incidencia

Diagrama de secuencia UML

En el siguiente diagrama de secuencia represento cómo interactúan entre sí el usuario, la interfaz gráfica, la cámara y la base de datos a lo largo del proceso de crear una incidencia y enviarla a las autoridades:

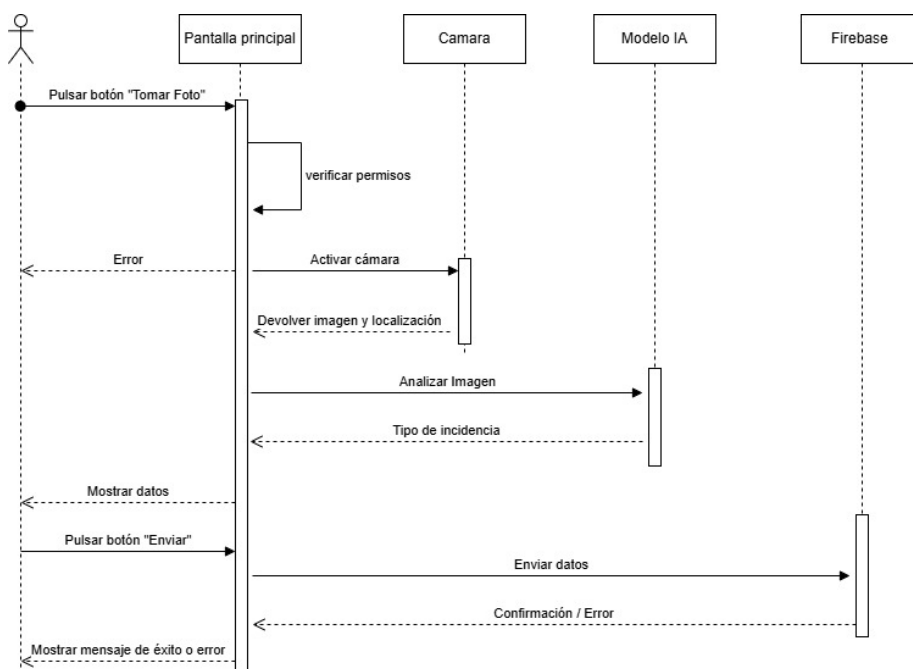


Figura C.12: Diagrama de secuencia UML crear y enviar incidencia

Consulta de las incidencias del usuario

Este aparatado corresponde al requisito **RF-09**, que especifica cualquier persona que tenga una cuenta y haya creado incidencias puede ver dichas incidencias, en el caso de uso de este requisito se explican las pasos que tiene que seguir el usuario así como las comprobaciones necesarias para que el registro se complete correctamente.

Para empezar, el usuario debe entrar en la pantalla principal de la aplicación y ahí tendrá un botón para poder las incidencias, este botón le llevará a una nueva pantalla donde podrá ver todas las incidencias, con su estado y el tipo de incidencia y luego un botón para ver mas en detalle las características de esta.

Diagrama de flujo

A continuación muestro el diagrama de flujo que resume de forma visual todos los pasos que se siguen durante el proceso de visualización de tus incidencias:

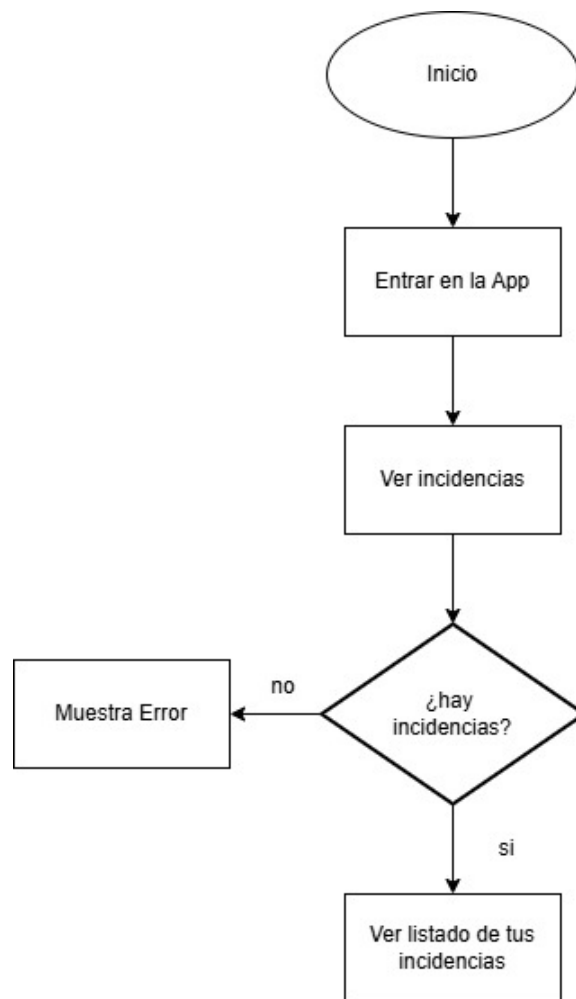


Figura C.13: Diagrama de flujo ver lista de incidencias

Diagrama de secuencia UML

En el siguiente diagrama de secuencia represento cómo interactúan entre sí el usuario, la interfaz gráfica y la base de datos a lo largo del proceso de ver el listado de incidencias del usuario:

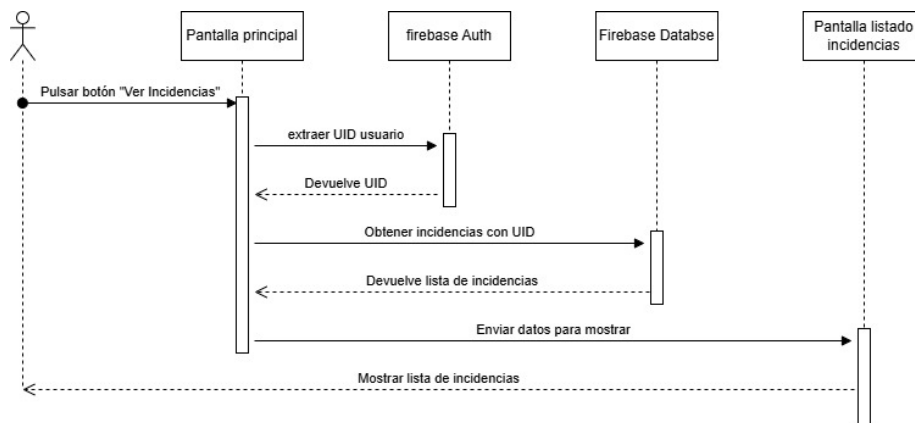


Figura C.14: Diagrama de secuencia UML ver lista de incidencia

Apéndice D

Documentación técnica de programación

D.1. Introducción

En este apéndice se recoge la parte más técnica de la aplicación, pensada para otros desarrolladores que quieran entender cómo está organizado el proyecto o que estén interesados en ampliarlo.

Se detalla la estructura del proyecto explicando qué contiene cada carpeta, y se incluyen las instrucciones necesarias para compilar y ejecutar la aplicación. También se describen las pruebas realizadas para comprobar que todo funciona correctamente, de forma que, si en el futuro se hacen modificaciones, estas pruebas puedan servir para verificar si los cambios han sido implementados correctamente o no.

D.2. Estructura de directorios

Aunque en el repositorio de GitHub [3] ya se detalla la estructura del proyecto en el archivo README, en esta sección también explico cómo está organizada mi aplicación.

El siguiente "árbol" muestra los directorios principales de mi aplicación. La carpeta **app** es la más importante ya que están todos los archivos y directorios necesarios para crear y poner en funcionamiento la APP. El resto de directorios como **ModelosIA** y **docs**, son directorios complementarios a la APP.

```
TFG-App-IA-IncidenciasViales/
|- CHANGELOG.md
|- README.md
|- code
|  |- android_app/
|  |  |- gradle/
|  |  |- app/
|  |  |- src/
|  |     |- androidTest/
|  |     |- main/
|  |        |- java/
|  |           |- com/
|  |              |- example/
|  |                 |- LoginActivity.java
|  |                    |- assets/
|  |                       |- labels.txt
|  |                       |- model_entrenado.tflite
|  |                          |- res/
|  |                             |- drawable/
|  |                             |- layout/
|  |                             |- mipmap-anydpi/
|  |                             |- mipmap-hdpi/
|  |                             |- mipmap-mdpi/
|  |                             |- mipmap-xhdpi/
|  |                             |- mipmap-xxhdpi/
|  |                             |- mipmap-xxxhdpi/
|  |                                |- values/
|  |                                |- values-night/
|  |                                   |- xml/
|  |                  |- test/
|  |- AI_model/
|  |-Firebase_funtions/
|  |-web_service/
|- docs/
|  |- research/
|  |- memoriaTFG/
```

Descripción de carpetas importantes:

- `code/android_app/app/src/main/java/com/example/`: Contiene el código fuente principal de la aplicación, donde se implementa la lógica y las funcionalidades principales.
- `code/android_app/app/src/main/res/layout/`: Archivos XML que definen las vistas y pantallas de la aplicación, especificando cómo se verá la interfaz de usuario.
- `code/android_app/app/src/main/res/drawable/`: Recursos gráficos como imágenes e íconos usados en la interfaz.
- `code/android_app/app/src/main/res/values/`: Recursos reutilizables como cadenas de texto, colores y estilos para mantener una interfaz coherente.
- `code/android_app/app/src/main/res/mipmap-*/`: Contiene los iconos de la aplicación en distintas resoluciones, necesarios para que se vean bien en todos los dispositivos Android.
- `code/android_app/app/src/main/res/xml/`: Archivos de configuración específicos, como los permisos o configuraciones de preferencias.
- `code/android_app/app/src/main/assets`: Contienen el modelo de IA que se usa en la aplicación y las labels que detecta.
- `code/android_app/app/src/androidTest/` y `app/src/test/`: Contienen las pruebas instrumentadas y pruebas unitarias para verificar el correcto funcionamiento del código.
- `code/android_app/gradle/`: Archivos de configuración necesarios para compilar y construir la aplicación correctamente. Aquí se definen las versiones, dependencias, plugins, etc.
- `docs/`: Documentación técnica y académica del proyecto, como investigaciones, diagramas y la memoria del TFG.
- `code/AI_model/`: Contiene todo lo necesario para poder entrenar el modelo de inteligencia artificial y transformarlo a formato TensorFlow Lite, que es el que se usa dentro de la app.
- `code/Firebase_funtions/`: Contiene todo lo necesario para poder enviar notificaciones a los dispositivos si el estado de la incidencia cambia.

- `code/web_service/`: Contiene todo lo necesario para poder crear una pagina web publica donde se pueda cambiar el estado de las incidencias y poder observar el listado de estas.
- `README.md`: Archivo con la explicación general del proyecto, cómo instalarlo, ejecutarlo y cualquier información básica relevante.
- `CHANGELOG.md`: Archivo donde se registran los cambios y actualizaciones del proyecto, útil para llevar un seguimiento del desarrollo.

D.3. Manual del programador

En este apartado explico cómo está hecha la aplicación para que cualquier desarrollador que quiera modificarla o ampliarla pueda hacerlo de forma sencilla y sin complicaciones.

Toda la lógica está escrita en Java, usando Android Studio como entorno de desarrollo. La estructura sigue el patrón clásico de una app Android con un Drawer Navigation, separando claramente los archivos de código, los recursos visuales (layouts, imágenes) y las configuraciones. En el apartado anterior se muestra un diagrama que indica dónde se encuentran los archivos del frontend y del backend.

Para entender bien el funcionamiento de la app, es recomendable empezar revisando el archivo `LoginActivity.java`, que actúa como punto de entrada. Desde ahí se puede navegar a las distintas pantallas y funcionalidades, como crear una cuenta, acceder a la pantalla principal tras iniciar sesión y usar las diferentes opciones disponibles.

Además, el proyecto incluye un modelo de IA entrenado aparte (guardado en la carpeta `AI_model`, dentro de la carpeta de `code`), que luego se convierte a formato `TensorFlow Lite` y se integra en la app para hacer predicciones sobre el tipo de incidencia. Este modelo `TensorFlow Lite` se introduce en la aplicación dentro de la carpeta `assets` y se utiliza principalmente en el `HomeFragment`.

Para el backend, he utilizado Firebase para autenticación, base de datos y almacenamiento. Las llamadas a Firebase están encapsuladas en funciones sencillas dentro del código Java, lo que facilita hacer cambios o incluso adaptar la app a otro backend si fuera necesario.

Aunque uso Firebase para guardar los datos en la nube, no es una base de datos SQL como las clásicas con tablas y relaciones. Firebase funciona con

un sistema NoSQL, que organiza la información en colecciones y documentos. Esto hace que sea más flexible y fácil de escalar. Por eso, en la app no hago consultas SQL, sino que trabajo directamente con esos documentos para leer o guardar la información.

D.4. Compilación, instalación y ejecución del proyecto

Aunque en el repositorio de GitHub ya se detalla cómo instalar y ejecutar el proyecto en el archivo README, en esta sección también explico cómo hacerlo.

Para poder usar la aplicación, lo primero que hay que hacer es obtener el código fuente. Esto se hace descargando el proyecto desde el repositorio [3], donde está todo lo necesario para que el proyecto funcione.

Después, hay que abrir el proyecto con Android Studio, que es el programa que reconocerá automáticamente los archivos necesarios, empezará a descargar las librerías y dependencias que la app necesita para funcionar correctamente.

Para probar la aplicación, es necesario contar con un dispositivo Android o un emulador. Android Studio facilita la creación y configuración de estos emuladores para que puedas probar la app sin necesidad de un dispositivo físico.

Cuando tengas el dispositivo o emulador listo, puedes compilar la aplicación desde Android Studio con un solo clic en el botón “Run” (o pulsando **Shift + F10**). Esto transformará el código en un programa instalable y automáticamente instalará la app en el dispositivo seleccionado, abriéndola para que puedas probarla.

Por último, te recomiendo probar las funciones principales de la aplicación para asegurarte de que todo funciona correctamente. Si tienes algún problema, revisa que el dispositivo o emulador esté bien configurado y que las versiones de las herramientas sean compatibles.

D.5. Pruebas del sistema

Para garantizar el correcto funcionamiento de la aplicación, se realizaron pruebas que cubren tanto la lógica interna como el comportamiento en dispositivos Android reales o emulados.

Pruebas UI (Android Tests)

Estas pruebas se ejecutan en dispositivos físicos o emuladores para verificar la interfaz de usuario y su interacción con el sistema operativo. Están ubicadas en `app/src/androidTest/` y utilizan el framework de pruebas de Android. Permiten validar la navegación, la carga correcta de datos y la respuesta a acciones del usuario.

Pruebas unitarias (Unit Tests)

Las pruebas unitarias se ejecutan localmente y verifican que funciones y métodos individuales en Java funcionen correctamente de forma aislada. Se encuentran en `app/src/test/` y validan la lógica de negocio y el procesamiento de datos sin necesidad de emulador o dispositivo físico.

Comparación entre modelos MobileNetV2 y EfficientNetB2

No se implementaron pruebas instrumentadas completas sobre la interfaz, por lo que se optó por comparar empíricamente los dos modelos entrenados con datos y configuraciones similares, que presentan diferencias en rendimiento y eficiencia.

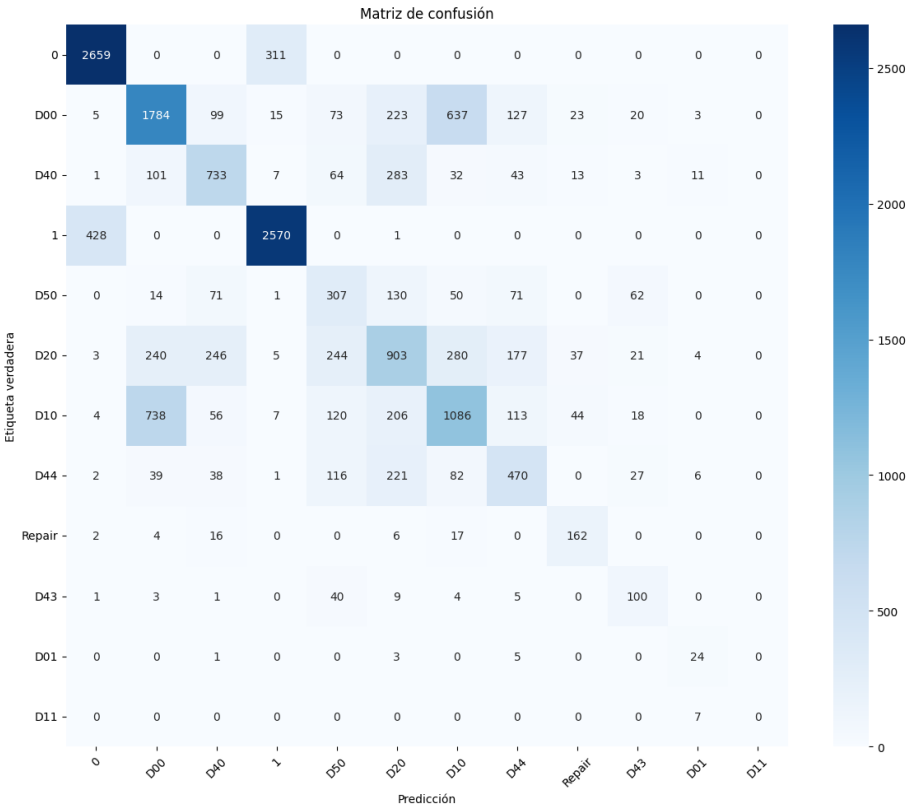


Figura D.1: Matriz confusión MobileNetV2

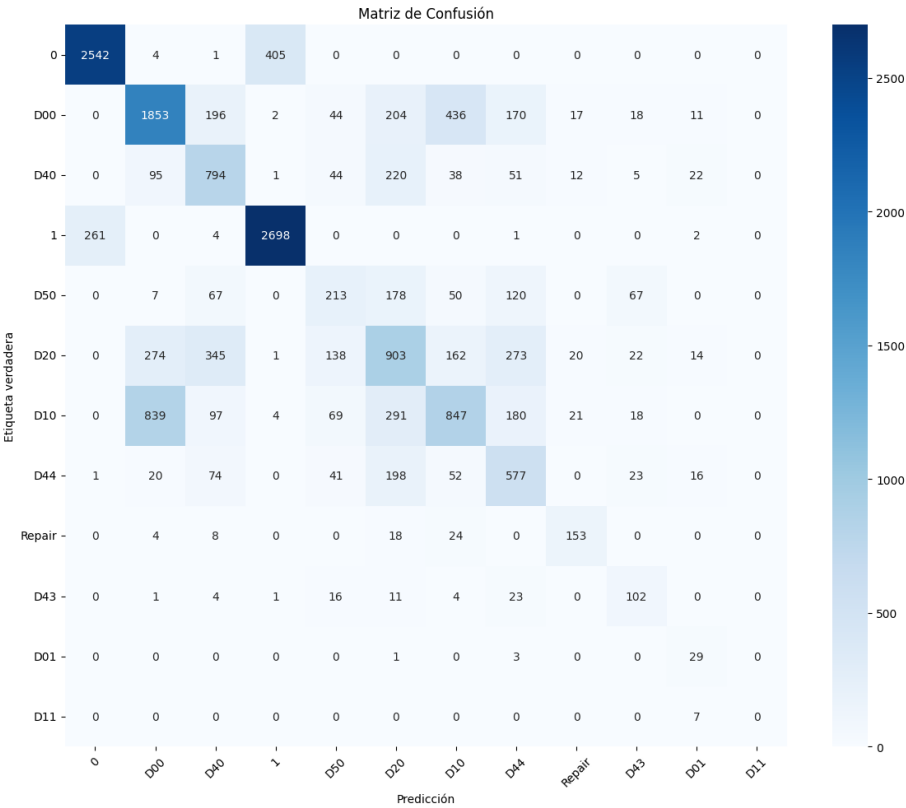


Figura D.2: Matriz confusión EfficientNetB2

Como se observa, las matrices de confusión son similares. MobileNetV2 acierta más en clases con mayor número de imágenes, mientras EfficientNetB2 mejora el acierto en clases más similares.

Clase	MobileNetV2			EfficientNetB2		
	Precisión	Recall	F1-score	Precisión	Recall	F1-score
0	0.8564	0.8953	0.8754	0.9066	0.8611	0.8833
D00	0.6103	0.5929	0.6015	0.5983	0.6279	0.6128
D40	0.5813	0.5678	0.5745	0.4994	0.6193	0.5529
1	0.8810	0.8570	0.8688	0.8670	0.9096	0.8878
D50	0.3185	0.4348	0.3677	0.3770	0.3034	0.3362
D20	0.4549	0.4181	0.4357	0.4461	0.4196	0.4325
D10	0.4963	0.4540	0.4742	0.5251	0.3580	0.4257
D44	0.4649	0.4691	0.4670	0.4127	0.5758	0.4808
Repair	0.5806	0.7826	0.6667	0.6861	0.7391	0.7116
D43	0.3984	0.6135	0.4831	0.4000	0.6296	0.4892
D01	0.4364	0.7273	0.5455	0.2871	0.8788	0.4328
D11	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Accuracy	0.6375			0.6382		
Macro avg	0.5066	0.5677	0.5300	0.5005	0.5769	0.5205
Weighted avg	0.6395	0.6375	0.6373	0.6406	0.6382	0.6349

Tabla D.1: Comparativa de métricas para MobileNetV2 y EfficientNetB2

Ambos modelos muestran desempeño global similar (accuracy 63.8%), pero con diferencias notables por clase:

- MobileNetV2 tiene mayor precisión en clases más frecuentes (0 y 1), con mejor F1-score.
- EfficientNetB2 mejora recall en varias clases (D00, D40, D44), detectando mejor ejemplos positivos, aunque con menor precisión en algunas.
- Ambas redes tienen bajo desempeño en clases minoritarias como D11.
- En tiempo de inferencia, MobileNetV2 es mucho más rápido (0.5s) que EfficientNetB2 (5s), factor clave para la elección según requisitos.

Apéndice E

Documentación de usuario

E.1. Introducción

En este apéndice se explica al usuario final cómo utilizar la aplicación móvil desarrollada. Esta aplicación está dirigida a ciudadanos que deseen reportar incidencias en la vía pública, con el fin de que las autoridades competentes puedan ser notificadas y actuar para resolverlas.

Se describen los requisitos necesarios, el proceso de instalación y el uso de las principales funcionalidades, con el objetivo de facilitar al usuario su utilización de forma sencilla y efectiva.

E.2. Requisitos de usuarios

Para utilizar la aplicación correctamente, el usuario debe cumplir con los siguientes requisitos:

- Disponer de un dispositivo Android con versión 8.0 (Oreo) o superior.
- Contar con conexión a Internet para acceder a las funcionalidades principales como el inicio de sesión, registro o sincronización de datos.
- Tener una cuenta de usuario registrada en el sistema.
- Poseer conocimientos básicos sobre el uso de aplicaciones móviles Android.

E.3. Instalación

Para instalar la aplicación en un dispositivo Android, se deben seguir los siguientes pasos:

1. Descargar el archivo APK desde el repositorio de GitHub[3] proporcionado por el autor del proyecto.



Figura E.1: Primer paso: descargar la app desde el repositorio

2. Activar la opción “Permitir la instalación de aplicaciones de fuentes desconocidas” en los ajustes del dispositivo.
3. Abrir el archivo APK descargado e iniciar el proceso de instalación.

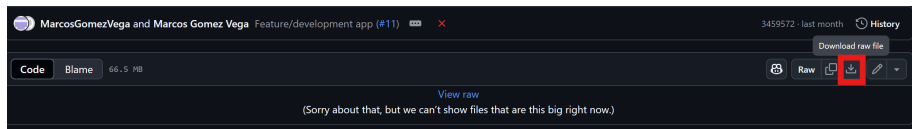


Figura E.2: Paso para instalar la aplicación en Android

4. Una vez instalada, acceder a la aplicación desde el menú de aplicaciones del dispositivo.

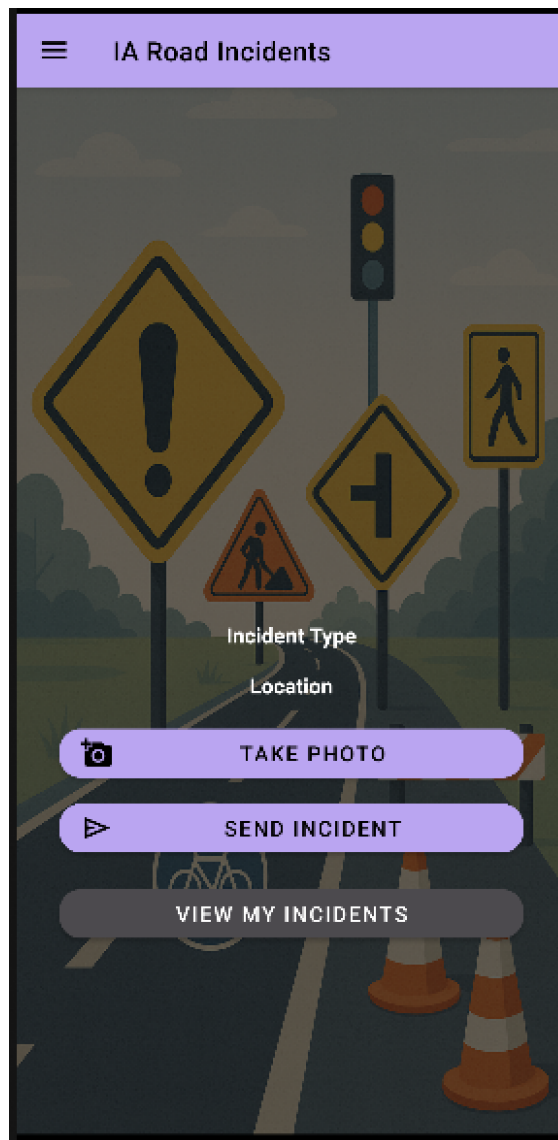
5. Conceder los permisos necesarios, como el acceso a la cámara y a la ubicación, para garantizar el correcto funcionamiento de la aplicación.

E.4. Manual del usuario

Guía para crear una incidencia

Una vez que el usuario ha iniciado sesión y se encuentra en la pantalla principal, puede proceder a reportar una incidencia en la vía pública siguiendo los siguientes pasos:

1. En la **pantalla principal**, pulsa el botón **“Tomar foto”** (ver Figura E.4).
2. Se abrirá la cámara del dispositivo. Captura una imagen clara del problema (por ejemplo, un bache, una grieta, una ondulación, etc.).
3. Una vez tomada la foto, se mostrará una vista previa. Si estás conforme con la imagen, pulsa **“Aceptar”**; si no, puedes volver a capturarla.
4. Tras aceptar la imagen, la aplicación iniciará un proceso de análisis automático mediante inteligencia artificial para detectar el tipo de incidencia. Esta clasificación se mostrará en el siguiente paso.
5. Una vez que la IA determine qué tipo de incidencia es, se mostrará la imagen, el tipo de incidencia y la localización en la pantalla principal.
6. Cuando hayas verificado que la IA ha detectado correctamente la incidencia, si no es así, puedes cambiarla manualmente en el desplegable que se encuentra a la derecha del tipo de incidencia. Luego podrás **“Enviar incidencia”**.
7. La aplicación confirmará que la incidencia ha sido enviada correctamente. Esta quedará registrada y podrás consultarla en la sección “Ver incidencias”.



Pantalla principal de la aplicación

Pantallas de inicio de sesión y registro

Para acceder a la aplicación, el usuario debe iniciar sesión o crear una cuenta nueva. Estas pantallas están diseñadas para facilitar el acceso de forma rápida y segura (ver Figuras E.4 y E.4).

Road Incident Detection



Email

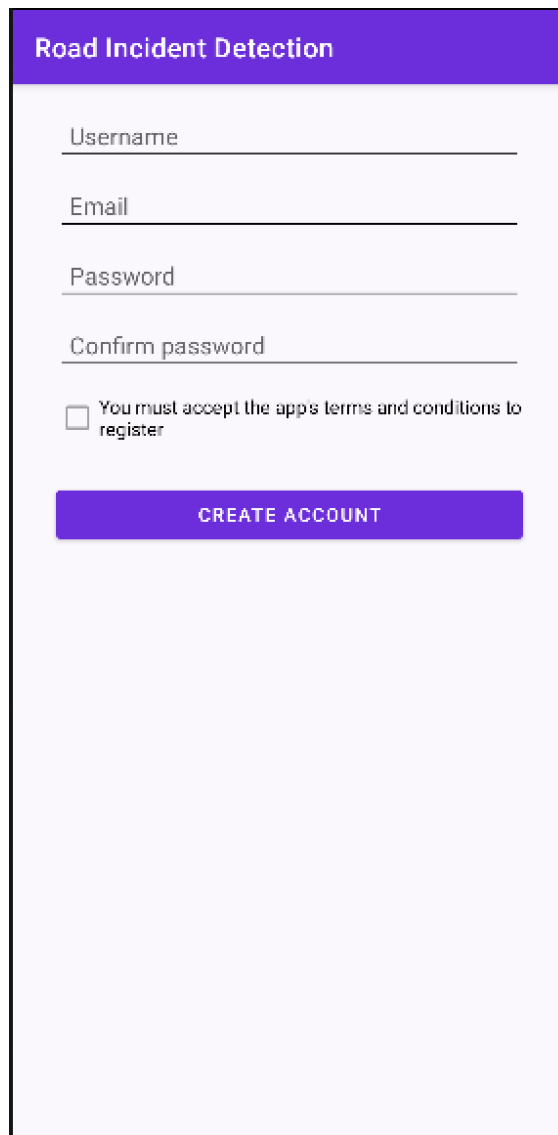
Password

LOG IN

CREATE ACCOUNT

 Continue with Google

Pantalla de inicio de sesión



Road Incident Detection

Username

Email

Password

Confirm password

☐ You must accept the app's terms and conditions to register

CREATE ACCOUNT

Pantalla de creación de cuenta

Consultar incidencias reportadas

La aplicación permite al usuario revisar todas las incidencias que ha reportado (Figura E.4). Desde esta pantalla se puede acceder a cada incidencia para ver detalles.

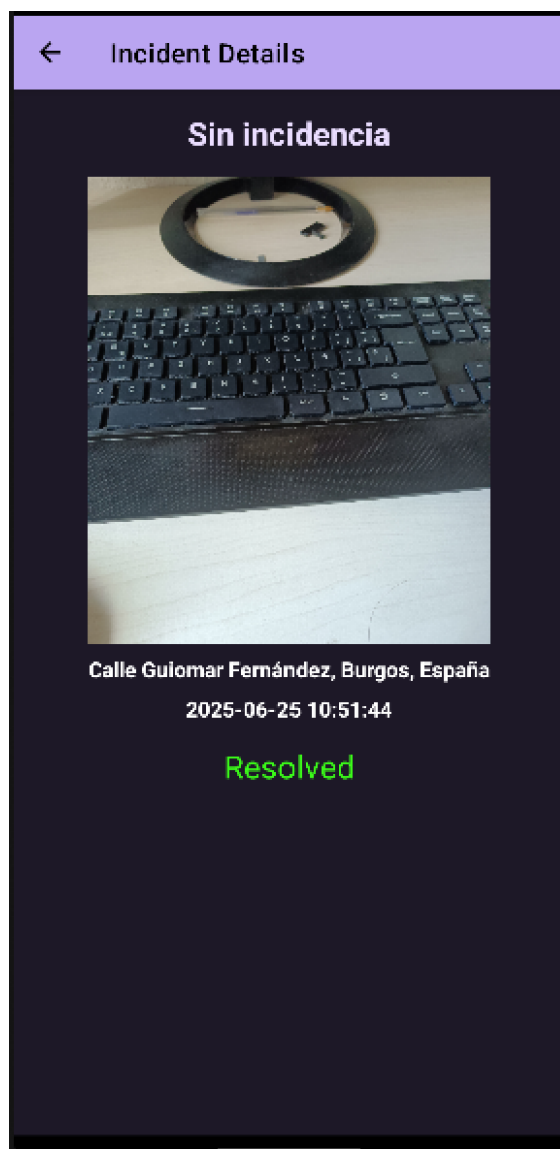
1. Pulsa el botón “**Ver incidencias**” en la pantalla principal o accede desde el menú lateral.
2. Se mostrará una lista con todas tus incidencias anteriores.
3. Selecciona una incidencia para ver detalles.



Pantalla con lista de incidencias reportadas

Detalle de una incidencia

Al consultar una incidencia en detalle, se muestra toda la información relevante: foto, tipo, localización y estado (Figura E.4).

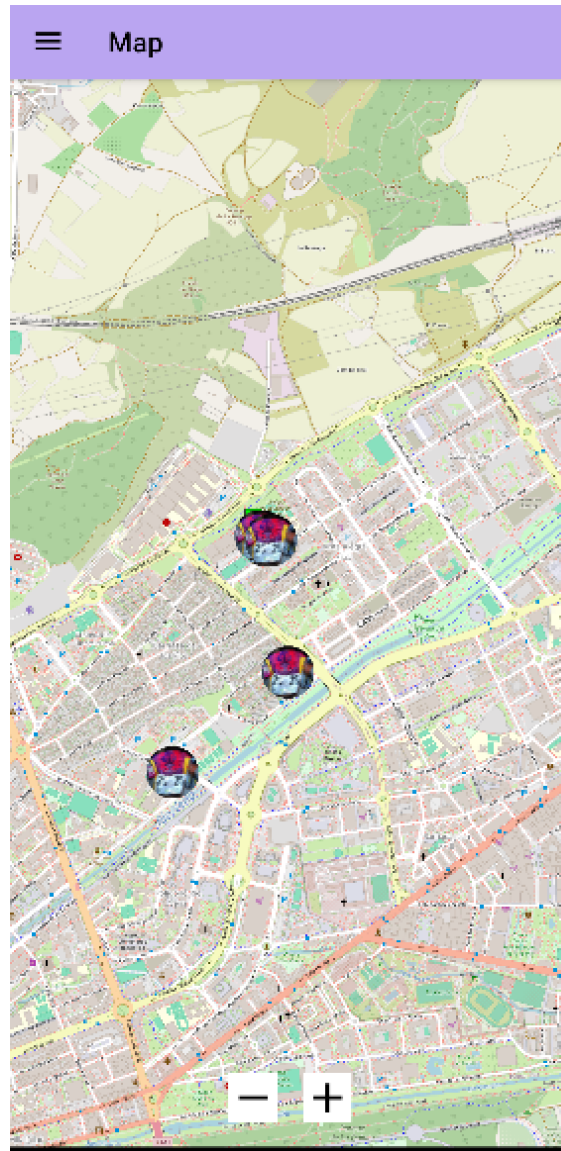


Detalle de una incidencia

Visualización en el mapa

La aplicación también permite visualizar todas las incidencias en un mapa interactivo (Figura E.4).

- Los colores de los marcadores indican el estado de cada incidencia.
- Con zoom cercano (menos de 3 km) se pueden ver incidencias de otros usuarios.

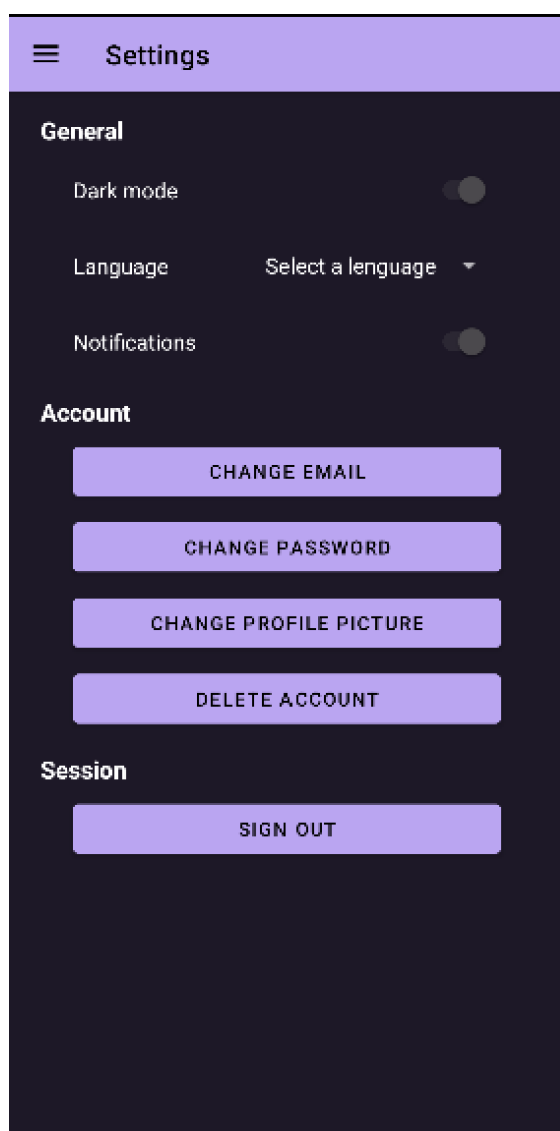


Mapa con las incidencias reportadas

Configuración de la aplicación

Desde la pantalla de configuración , accesible desde el menú lateral, el usuario puede personalizar:

- Modo oscuro
- Idioma
- Notificaciones
- Cambiar correo y contraseña
- Eliminar cuenta
- Cerrar sesión



Pantalla de configuración

E.5. Recomendaciones y soporte

Se recomienda al usuario mantener la aplicación actualizada y revisar regularmente las notificaciones para estar informado del estado de sus incidencias.

Para soporte técnico o dudas, el usuario puede consultar el repositorio de GitHub del proyecto^[3] o contactar al desarrollador responsable.

Anexo de sostenibilización curricular

F.1. Introducción

La sostenibilidad ha ido ganando importancia en mi vida personal a lo largo de los años. Gracias a mi paso por la universidad y, en particular, al desarrollo del TFG, he podido interiorizarla también como una competencia profesional. En este anexo reflexiono sobre cómo he aplicado criterios de sostenibilidad durante la elaboración de mi TFG y cómo se han materializado algunas de las competencias básicas descritas por la CRUE [1].

Mi TFG consiste en el diseño de una *aplicación móvil orientada al reporte de incidencias*. Desde el inicio del proyecto entendí que debía incorporar una mirada ética, inclusiva y responsable. Más allá de los aspectos técnicos, quise desarrollar una herramienta útil para la comunidad, eficiente en el uso de recursos y accesible para todos los perfiles de usuario.

F.2. Conciencia del impacto social y ambiental

Uno de los aprendizajes más significativos ha sido comprender que cualquier proyecto, incluso uno tecnológico como este, puede tener un impacto relevante en la sociedad y el medio ambiente. En mi caso, reflexioné sobre el consumo de recursos asociado al uso de la aplicación y decidí utilizar tecnologías ligeras, que no demandaran excesiva potencia de procesamiento

ni almacenamiento. Esto reduce el impacto energético del sistema, tanto en los dispositivos móviles como en los servidores.

También me esforcé por garantizar la accesibilidad. Integré principios de diseño universal para que cualquier persona, independientemente de su edad, nivel de experiencia o capacidades físicas, pueda utilizar la aplicación. Este tipo de decisiones, aunque no siempre visibles a simple vista, reflejan el compromiso con una sostenibilidad social que promueva la equidad.

F.3. Enfoque sistémico y diseño inclusivo

Conforme avanzaba en el desarrollo, me di cuenta de que el éxito del proyecto no dependía solo de su funcionamiento técnico, sino de su capacidad para integrarse en un entorno real. Por eso, amplíé mi enfoque para considerar no solo al usuario final, sino también a las instituciones y colectivos que podrían beneficiarse de una herramienta así.

El diseño pasó a ser más abierto y adaptable, permitiendo la incorporación de nuevos módulos o integraciones según las necesidades de cada municipio o comunidad. Esta visión sistémica está en línea con lo que la CRUE plantea como una competencia clave: la capacidad de abordar los problemas desde un enfoque holístico y transversal.

F.4. Participación y aprendizaje transversal

Aunque el TFG es un trabajo individual, durante el proceso recurrí al apoyo de personas con diferentes perfiles: desde compañeros con formación en diseño gráfico y accesibilidad digital, hasta usuarios potenciales que me ofrecieron sus opiniones y sugerencias, incluyendo profesorado que me ha ayudado en momentos clave. Esta interacción me ayudó a desarrollar habilidades para trabajar en contextos interdisciplinarios y a valorar diferentes puntos de vista antes de tomar decisiones.

F.5. Ética profesional y compromiso personal

Una de las lecciones más valiosas que me llevo de este proyecto es la importancia de la ética en la toma de decisiones. Desde el principio tuve claro que no bastaba con cumplir los requisitos técnicos: debía reflexionar también sobre las implicaciones sociales de mi trabajo.

Por ejemplo, me preocupaba el uso que se pudiera hacer de los datos recopilados a través de la aplicación. Por ello, establecí mecanismos que priorizan la privacidad, la transparencia y el control del usuario sobre su propia información. Este tipo de decisiones no solo están alineadas con buenas prácticas profesionales, sino también con los valores promovidos por la CRUE: responsabilidad, justicia y respeto por los derechos humanos.

F.6. Conclusión

El desarrollo de este TFG me ha permitido poner en práctica competencias clave en sostenibilidad: desde el enfoque sistémico, hasta la participación y la responsabilidad ética. He aprendido que la sostenibilidad no es un añadido, sino una forma de entender y ejercer la profesión de manera coherente con los desafíos del presente y del futuro.

Esta experiencia ha reforzado mi compromiso con un ejercicio profesional más consciente y sensible al impacto social y ambiental de la tecnología. Espero seguir aplicando estos principios en mis futuros proyectos, y contribuir desde mi campo al Desarrollo Humano Sostenible.

Bibliografía

- [1] CRUE Conferencia de Rectores de las Universidades Españolas. Directrices para la sostenibilización curricular en la educación superior, 2012.
- [2] Marcos Gómez Vega. Microsft planner, 2025.
- [3] Marcos Gómez Vega. Repositorio del proyecto en github, 2025.